UNIT-III
LINEAR MODELS

Linear Models:

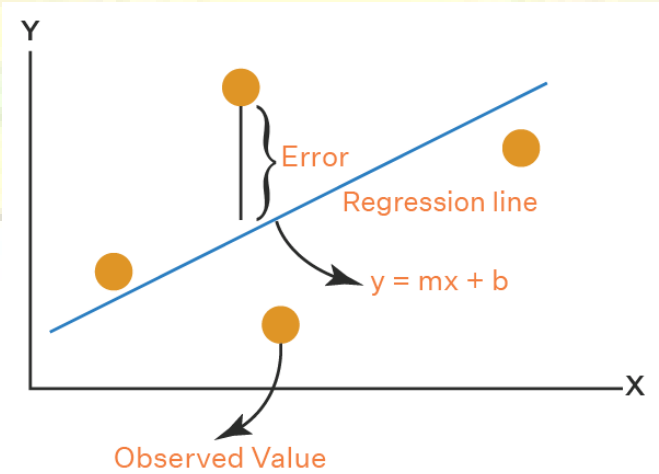
- Linear models are parametric i.e., they have a fixed form with a small number of numeric parameters that need to be learned from data. This different from tree or rule models, where the structure of the model (e.g., which features to use in the tree, and where) is not fixed in advance.
- Linear models are stable, means that small variations in the training data have only limited impact on the learned model. Tree models tend to vary more with the training data, as the choice of a different split at the root of the tree typically means that the rest of the tree is different as well.
- Linear models are less likely to overfit the training data than some other models, largely because they have relatively few parameters. Note, sometimes lead to underfitting: e.g., imagine you are learning where the border runs between two countries from labelled samples, then a linear model is unlikely to give a good approximation.

The last two points says that linear models have low variance but high bias. Such models are often preferable when you have limited data and want to avoid overfitting. High variance–low bias models such as decision trees are preferable if data is abundant but underfitting is a concern.

Linear models exist for all predictive tasks, including classification, probability estimation and regression. Linear regression, in particular, is a well-studied problem that can be solved by the least-squares method.

Least Squares method

- Least square method is the process of finding a regression line or best-fitted line for any data set that is described by an equation.
- This method requires reducing the sum of the squares of the residual parts of the points from the curve or line and the trend of outcomes is found quantitatively.
- The method of curve fitting is seen while regression analysis and the fitting equations to derive the curve is the least square method.
- The least-squares method is a statistical method used to find the line of best fit of the form of an equation such as y = mx + b to the given data.
- The curve of the equation is called the regression line.

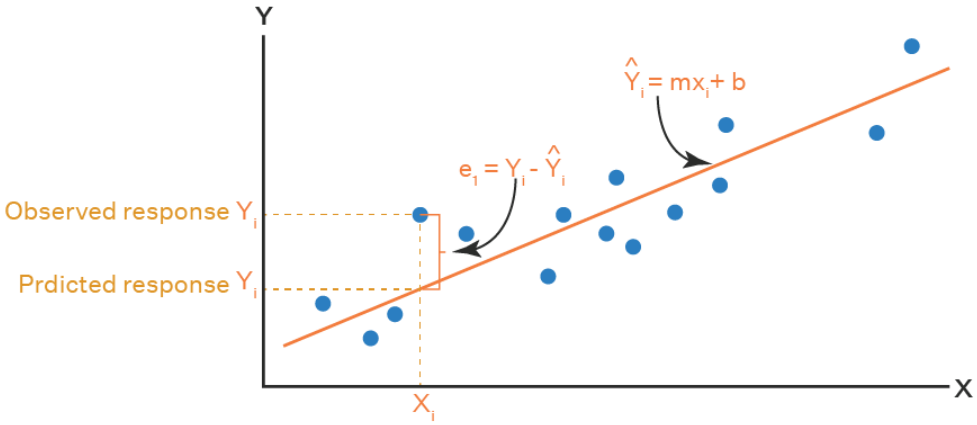


- The least-squares method is used to predict the behavior of the dependent variable with respect to the independent variable.
- The sum of the squares of errors is called variance and helps in finding the variation in observed data.

- Our main objective in this method is to minimize the sum of the squared errors as much as possible. This is the reason this method is called the least-squares method.
- This method is often used in data fitting where the best fit result is assumed to reduce the sum of squared errors that is considered to be the difference between the observed values and corresponding fitted value.

$$\sum_{i=1}^{n}(y_i - (mx_i + b))^2$$

- For example, we have 4 data points and using this method we arrive at the following graph.



- The two basic categories of least-square problems are ordinary or linear least squares and nonlinear least squares.

Limitations for Least Square Method

- This method exhibits only the relationship between the two variables. All other causes and effects are not taken into consideration.
- It is unreliable when data is not evenly distributed.
- It is very sensitive to outliers, which can skew the results in analysis.

Multivariate Linear Regression

- Multivariate Regression is a supervised machine learning algorithm involving multiple data variables for analysis.
- Multivariate regression is an extension of multiple regression with one dependent variable and multiple independent variables. Based on the number of independent variables, we try to predict the output.
- Multivariate regression allows one to have a different view of the relationship between various variables from all the possible angles.
- Multivariate regression technique can be implemented efficiently with the help of matrix operations.
- Multivariate regression tries to find out a formula that can explain how factors in variables respond simultaneously to changes in others.
- There are numerous areas where multivariate regression can be used. Let's look at some examples to understand multivariate regression better.
  1. A doctor investigating the relationship between health measures (cholesterol, blood pressure, weight) and eating habits.
  2. An agriculture scientist wants to predict the total crop yield expected for the summer. He collected details of the expected amount of rainfall, fertilizers to be used, and soil conditions.

3. A property dealer determining housing prices based on factors like house size in square feet, number of bedrooms, age, location of the house, amenities available, or not.
4. If an organization wants to predict how much it has to pay while hiring a new employee, they will take into account many details such as education level, number of experience, job location, has niche skill or not.
5. A company wants to predict the electricity bill of an apartment, based on number of flats, the number of appliances in usage, the number of people at home, etc.

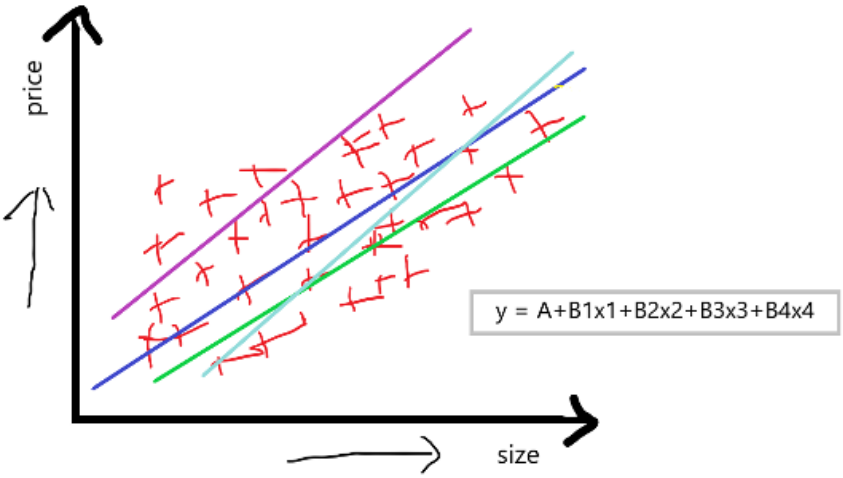
- The above examples use Multivariate regression, where we have many independent variables and a single dependent variable.
- Below is the generalized equation for the multivariate regression model-

$$y = \beta_0 + \beta_1.x_1 + \beta_2.x_2 + \ldots.. + \beta_n.x_n$$

Where n represents the number of independent variables, $\beta_0 \sim \beta_n$ represents the coefficients, and $x_1 \sim x_n$ is the independent variable.

- The multivariate model helps us in understanding and comparing coefficients across the output. Here, the small cost function makes Multivariate linear regression a better model.
- Cost of Multiple Linear regression:

$$MSE = \frac{1}{2m}\sum(h_\theta(x)^{(i)} - y^i)^2$$



Advantages:-

- The most important advantage of Multivariate regression is it helps us to understand the complex relationships among multiple variables present in the dataset.

Disadvantages:-

- It assumes a linear relationship (may not hold for all data).
- Multivariate techniques are a bit complex and require a high-level of mathematical calculation.
- The results of the model are better for larger datasets only.
- It is sensitive to outliers, which makes the results analysis complex.

Regularized Regression

We have just seen a situation in which least-squares regression can become unstable: i.e., highly dependent on the training data. Instability is a manifestation of a tendency to overfit. Regularization is a general method to avoid such overfitting by applying additional constraints to the weight vector. A common approach is to make sure the weights are, on average, small in magnitude: this is referred to as shrinkage.

- Overfitting is a phenomenon that occurs when a Machine Learning model is constrained to the training set and not able to perform well on unseen data. That is when our model learns the noise in the training data as well.
- Bias refers to the errors which occur when we try to fit a statistical model on real-world data which does not fit perfectly well on some mathematical model.
- Variance implies the error value that occurs when we try to make predictions by using data that is not previously seen by the model.
- Regularization is a technique used to reduce errors by fitting the function appropriately on the given training set and avoiding overfitting. The commonly used regularization techniques are :
    1. Lasso Regularization – L1 Regularization
    2. Ridge Regularization – L2 Regularization
    3. Elastic Net Regularization – L1 and L2 Regularization

1. Lasso Regularization – L1 Regularization

- A regression model which uses the L1 Regularization technique is called LASSO (Least Absolute Shrinkage and Selection Operator) regression.
- Lasso Regression adds the "*absolute weight*" of the coefficient as a penalty term to the loss function (L).
- Lasso regression also helps us achieve feature selection by penalizing the weights to approximately equal to zero if that feature does not serve any purpose in the model.
- Hence, the Lasso regression can help us to reduce the overfitting in the model as well as the feature selection.

$$Cost = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{m}|w_i|$$

where, m – Number of Features

n – Number of Examples

- One of the key advantages of Lasso regression is its ability to handle datasets with a large number of predictors efficiently.

2. Ridge Regularization – L2 Regularization

- A regression model that uses the L2 regularization technique is called Ridge regression.
- In Ridge regression a small amount of bias is introduced so that we can get better long-term predictions.
- The amount of bias added to the model is called Ridge Regression penalty.
- Ridge regression adds the "squared weight/magnitude" of the coefficient as a penalty term to the loss function (L).
- A general linear or polynomial regression will fail if there is high collinearity (multicollinearity) between the independent variables, so to solve such problems, Ridge regression can be used.
- It helps to solve the problems if we have more parameters than samples.

$$Cost = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{m}(w_i)^2$$

Key Difference between Ridge Regression and Lasso Regression

- ***Ridge regression*** is mostly used to reduce the overfitting in the model, and it includes all the features present in the model. It reduces the complexity of the model by shrinking the coefficients.
- ***Lasso regression*** helps to reduce the overfitting in the model as well as feature selection.

3. Elastic Net Regularization – L1 and L2 Regularization

- This model is a combination of L1 as well as L2 regularization.

- It means that we can add the absolute norm of the weights as well as the squared measure of the weights using an extra hyperparameter that controls the ratio of the L1 and L2 regularization.
- Elastic Net provides a balance between feature selection (Lasso) and coefficient shrinkage (Ridge), offering flexibility in controlling model complexity.

$$Cost = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\left((1-\propto)\sum_{i=1}^{m}|w_i| + \propto\sum_{i=1}^{m}{w_i}^2\right)$$

Using Least Square regression for Classification

- Least Squares Regression is a statistical method commonly used in machine learning for analyzing and modeling data. However, it is primarily associated with regression tasks, not classification tasks.
- In classification, the target is a categorical value ("yes/no," "red/blue/green," "spam/not spam," etc.). Regression involves numerical, continuous values as a target.
- Least Squares Regression (LSR) aims to find the line of best fit (often a straight line) that minimizes the sum of the squared residuals (the differences between actual values and predicted values) between the independent variable(s) and the dependent variable.
- It's widely used for predicting continuous numeric outcomes (such as house prices, temperature, or stock prices).

Why Least Squares Regression Is Not Suitable for Classification:

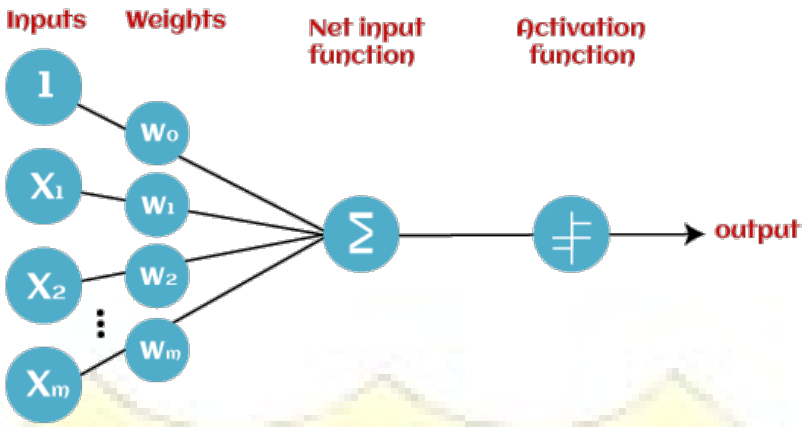
- LSR assumes a linear relationship between variables, which doesn't align well with classification problems.
- In classification, we need to assign data points to specific classes (e.g., binary classes like "yes" or "no").
- LSR doesn't naturally handle this discrete assignment.
- So, when dealing with classification, focus on algorithms that directly address class assignments and probabilities.

Alternatives for Classification:

- ***Logistic Regression:*** Despite the name, it's a classification algorithm. It models the probability of belonging to a particular class.
- ***Support Vector Machines (SVM):*** Powerful for both binary and multiclass classification.
- ***Decision Trees and Random Forests:*** Tree-based methods that handle classification naturally.
- ***Neural Networks:*** Deep learning models capable of handling complex classification tasks.

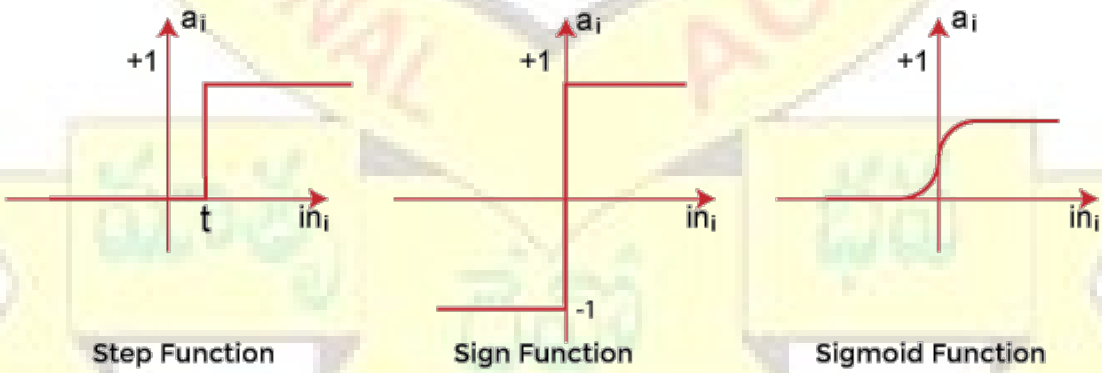
Perceptron

- Perceptron is Machine Learning algorithm for supervised learning of various binary classification tasks. It can learn the linearly separable patterns.
- Further, Perceptron is also understood as an Artificial Neuron or neural network unit that helps to detect certain input data computations in business intelligence.
- It was introduced by Frank Rosenblatt in 1957s. It is the simplest type of feed-forward neural network, consisting of a single layer of input nodes that are fully connected to a layer of output nodes.
- However, it is a supervised learning algorithm of binary classifiers. Hence, we can consider it as a single-layer neural network with four main parameters, i.e., ***input values, weights and Bias, net sum,*** and an ***activation function***.



Basic Components of Perceptron

- Mr. Frank Rosenblatt invented the perceptron model as a binary classifier which contains three main components. These are as follows:
  1. ***Input Nodes or Input Layer:*** It is the primary component of Perceptron which accepts the initial data into the system for further processing. Each input node contains a real numerical value.
  2. ***Wight and Bias:*** Weight parameter of Perceptron components represents the strength of the connection between units. Weight is directly proportional to the strength of the associated input neuron in deciding the output. Further, Bias can be considered as the line of intercept in a linear equation.
  3. ***Activation Function:*** It is the final and important component that help to determine whether the neuron will fire or not. Activation Function can be considered primarily as a step function. Types of Activation functions:
     - ✓ Sign function
     - ✓ Step function, and
     - ✓ Sigmoid function



Types of Perceptron Models

Based on the layers, Perceptron models are divided into two types. These are as follows:

1. Single-layer Perceptron Model
2. Multi-layer Perceptron model

1. Single Layer Perceptron Model:

- A single-layered perceptron model consists feed-forward network and also includes a threshold transfer function inside the model.
- The main objective of the single-layer perceptron model is to analyze the linearly separable

objects with binary outcomes.

- In a single layer perceptron model, its algorithms do not contain recorded data, so it begins with inconstantly allocated input for weight parameters. Further, it sums up all inputs (weight). After adding all inputs, if the total sum of all inputs is more than a pre-determined value, the model gets activated and shows the output value as +1.
- If the outcome is same as pre-determined or threshold value, then the performance of this model is stated as satisfied, and weight demand does not change. However, this model consists of a few discrepancies triggered when multiple weight inputs values are fed into the model. Hence, to find desired output and minimize errors, some changes should be necessary for the weights input.
- "Single-layer perceptron can learn only linearly separable patterns."

2. ***Multi-Layered Perceptron Model:***

- Like a single-layer perceptron model, a multi-layer perceptron model also has the same model structure but has a greater number of hidden layers.
- The multi-layer perceptron model is also known as the Backpropagation algorithm, which executes in two stages as follows:
  - ✓ *Forward Stage:* Activation functions start from the input layer in the forward stage and terminate on the output layer.
  - ✓ *Backward Stage:* In the backward stage, weight and bias values are modified as per the model's requirement. In this stage, the error between actual output and demanded originated backward on the output layer and ended on the input layer.

Hence, a multi-layered perceptron model has considered as multiple artificial neural networks having various layers in which activation function does not remain linear, similar to a single layer perceptron model. Instead of linear, activation function can be executed as sigmoid, TanH, ReLU, etc., for deployment.

A multi-layer perceptron model has greater processing power and can process linear and non-linear patterns. Further, it can also implement logic gates such as AND, OR, XOR, NAND, NOT, XNOR, NOR

Advantages of Multi-Layer Perceptron:

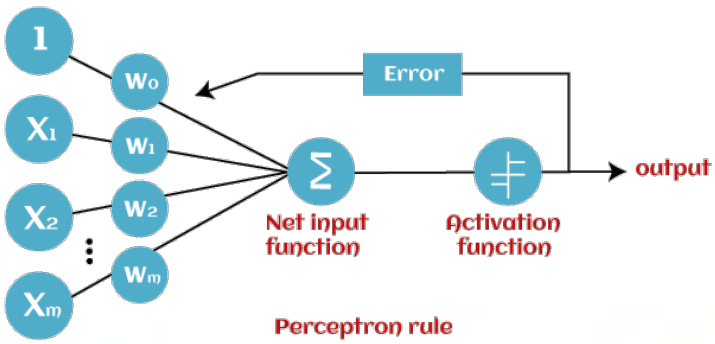
- ✓ A multi-layered perceptron model can be used to solve complex non-linear problems.
- ✓ It works well with both small and large input data.
- ✓ It helps us to obtain quick predictions after the training.
- ✓ It helps to obtain the same accuracy ratio with large as well as small data.

Disadvantages of Multi-Layer Perceptron:

- ✓ In Multi-layer perceptron, computations are difficult and time-consuming.
- ✓ In multi-layer Perceptron, it is difficult to predict how much the dependent variable affects each independent variable.
- ✓ The model functioning depends on the quality of the training.

How does Perceptron work?

- In Machine Learning, Perceptron is considered as a single-layer neural network that consists of four main parameters named ***input values (Input nodes), weights and Bias, net sum,*** and an ***activation function***.
- The perceptron model begins with the multiplication of all input values and their weights, then adds these values together to create the weighted sum. Then this weighted sum is applied to the activation function 'f' to obtain the desired output. This activation function is also known as the step function and is represented by 'f'.



- This step function or Activation function plays a vital role in ensuring that output is mapped between required values (0,1) or (-1,1). It is important to note that the weight of input is indicative of the strength of a node. Similarly, an input's bias value gives the ability to shift the activation function curve up or down.

Perceptron model works in two important steps as follows:

Step-1: In the first step first, multiply all input values with corresponding weight values and then add them to determine the weighted sum. Mathematically, we can calculate the weighted sum as follows:

$$\sum \mathbf{w_i * x_i = x_1 * w_1 + x_2 * w_2 + \ldots w_n * x_n = X^T W}$$

Add a special term called bias 'b' to this weighted sum to improve the model's performance.

$$\sum \mathbf{w_i * x_i + b}$$

Step-2: In the second step, an activation function is applied with the above-mentioned weighted sum, which gives us output either in binary form or a continuous value as follows:

$$\mathbf{h(z) = Y = f\left(\sum w_i * x_i + b\right)}$$

$$f_{W,b}(X) = h(XW + b)$$

Where, X is the input W is the weight for each inputs neurons and b is the bias and h is the step function.

$$w_{i,j} = w_{i,j} + \eta\,(y_j - \hat{y}_j)x_i$$

Here, $w_{i,j}$ is the weight between the ith input and j^{th} output neuron, x_i is the ith input value, y_j and $\hat{y}_j$ is the jth actual and predicted value and η is the learning rate.

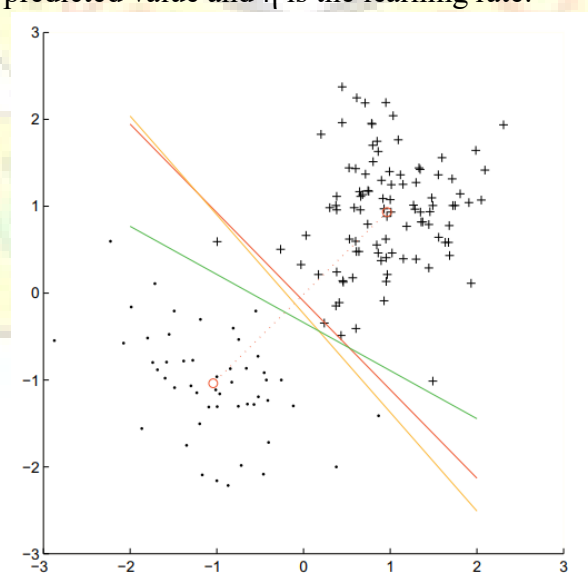
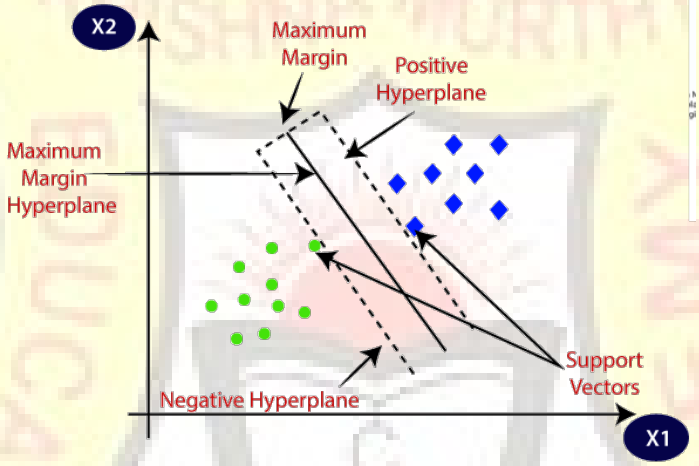


Fig:- The basic linear classifier in red, the least-squares classifier in orange and the perceptron in green.

- Notice that the perceptron perfectly separates the training data, but its heuristic approach may lead to overfitting in certain situations.

Support Vector Machines

- Support Vector Machines (SVM) are powerful supervised learning algorithms used for both classification and regression tasks in machine learning. However, it is best suited for Classification problems in Machine Learning.
- The main objective of the SVM algorithm is to find the optimal ***hyperplane*** in an N-dimensional space that can separate the data points in different classes in the feature space.
- The hyperplane/hard margin tries that the margin between the closest points of different classes should be as maximum as possible.
- The dimension of the hyperplane depends upon the number of features.
- If the number of input features is two, then the hyperplane is just a line. If the number of input features is three, then the hyperplane becomes a 2-D plane. It becomes difficult to imagine when the number of features exceeds three.



- SVMs can be used for a variety of tasks, such as text classification, image classification, spam detection, handwriting identification, gene expression analysis, face detection, and anomaly detection.
- SVMs are adaptable and efficient in a variety of applications because they can manage high-dimensional data and nonlinear relationships.

Support Vector Machine Terminology

- ***Hyperplane:*** Hyperplane is the decision boundary that is used to separate the data points of different classes in a feature space. In the case of linear classifications, it will be a linear equation i.e. wx+b = 0.
- ***Support Vectors:*** Support vectors are the closest data points to the hyperplane, which makes a critical role in deciding the hyperplane and margin.
- ***Margin:*** Margin is the distance between the support vector and hyperplane. The main objective of the support vector machine algorithm is to maximize the margin. The wider margin indicates better classification performance.
- ***Kernel:*** Kernel is the mathematical function, which is used in SVM to map the original input data points into high-dimensional feature spaces, so, that the hyperplane can be easily found out even if the data points are not linearly separable in the original input space. Some of the common kernel functions are linear, polynomial, radial basis function(RBF), and sigmoid.
- ***Hard Margin:*** The maximum-margin hyperplane or the hard margin hyperplane is a hyperplane that properly separates the data points of different categories without any misclassifications.
- ***Soft Margin:*** When the data is not perfectly separable or contains outliers, SVM permits a soft margin technique. Each data point has a slack variable introduced by the soft-margin SVM

formulation, which softens the strict margin requirement and permits certain misclassifications or violations. It discovers a compromise between increasing the margin and reducing violations.

- ***Hinge Loss:*** A typical loss function in SVMs is hinge loss. It punishes incorrect classifications or margin violations. The objective function in SVM is frequently formed by combining it with the regularization term.

Types of Support Vector Machine

Based on the nature of the decision boundary, Support Vector Machines (SVM) can be divided into two main parts:

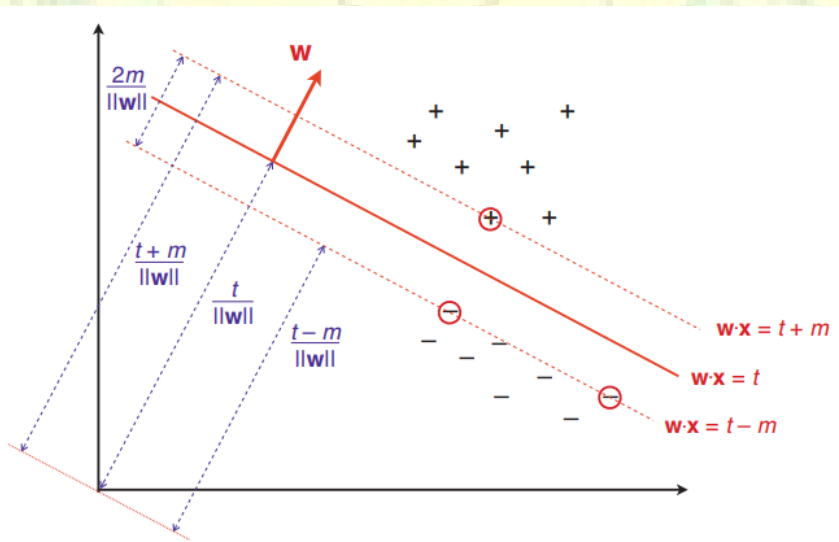
- ***Linear SVM:*** Linear SVM is used for linearly separable data. They use a linear decision boundary to separate the data points of different classes. When the data can be precisely linearly separated, linear SVMs are very suitable. This means that a single straight line (in 2D) or a hyperplane (in higher dimensions) can entirely divide the data points into their respective classes. A hyperplane that maximizes the margin between the classes is the decision boundary.
- ***Non-Linear SVM:*** Non-Linear SVM is used for non-linearly separated data i.e., Non-Linear SVM can be used to classify data when it cannot be separated into two classes by a straight line (in the case of 2D). By using kernel functions, nonlinear SVMs can handle nonlinearly separable data. The original input data is transformed by these kernel functions into a higher-dimensional feature space, where the data points can be linearly separated. A linear SVM is used to locate a nonlinear decision boundary in this modified space.

How does SVM work?

- Consider a binary classification problem with two classes, labeled as +1 and -1. We have a training dataset consisting of input feature vectors X and their corresponding class labels Y.
- The equation for the linear hyperplane can be written as: $w^T x + b = 0$
  Where, W is the vector represents the normal vector to the hyperplane. i.e., the direction perpendicular to the hyperplane. The parameter b in the equation represents the offset or distance of the hyperplane from the origin along the normal vector w.
- The distance between a data point x_i and the decision boundary can be calculated as:

$$d_i = \frac{w^T x_i + b}{||w||}$$

  where ||w|| represents the Euclidean norm of the weight vector w. Euclidean norm of the normal vector W.
- For Linear SVM classifier : $\hat{y} = \begin{cases} 1 : & w^T x + b \geq 0 \\ 0 : & w^T x + b < 0 \end{cases}$



- One reasonable choice as the best hyperplane is the one that represents the largest separation or margin between the two classes.
- So we choose the hyperplane whose distance from it to the nearest data point on each side is maximized. If such a hyperplane exists it is known as the maximum-margin hyperplane/hard margin.

Advantages of Hard Margin

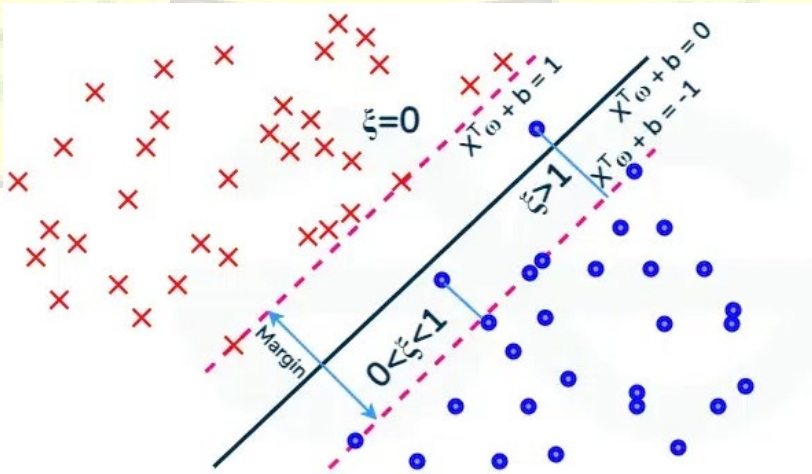
- ***Guaranteed Separation:*** Hard margin SVM ensures that the classes are perfectly separated, leading to optimal generalization performance when the training data is linearly separable.
- ***Simplicity:*** The optimization problem in hard margin SVM is well-defined and has a unique solution, making it computationally efficient.

Disadvantages of Hard Margin

- ***Sensitivity to Outliers:*** Hard margin SVM is highly sensitive to outliers or noisy data points. Even a single mislabeled point can significantly affect the position of the decision boundary and lead to poor generalization on unseen data.
- ***Not Suitable for Non-linear Data:*** When the data is not linearly separable, hard margin SVM fails to find a valid solution, rendering it impractical for many real-world datasets.

Soft Margin SVM

- Soft Margin SVM introduces flexibility by allowing some margin violations (misclassifications) to handle cases where the data is not perfectly separable.
- Suitable for scenarios where the data may contain noise or outliers.
- It introduces a penalty term for misclassifications, allowing for a trade-off between a wider margin and a few misclassifications.
- This adaptability is managed by a factor called C, also called the "regularization parameter," that controls the trade-off between maximizing the margin and minimizing the classification error.
- The soft margin SVM follows a somewhat similar optimization procedure with a couple of differences. First, need to minimize the misclassification error. Second, to minimize the error, we should define a loss function.
- A common loss function used for soft margin is the hinge loss.
- The difference between the primal problem and the one for the hard margin is the addition of slack variables. The new slack variables (ξ_i in the figure below) add flexibility for misclassifications of the model:



Mathematically, decision function of a soft margin SVM can defined as: $y_i(x^T w_i + b) \geq 1 - \xi_i$

Where:

- ✓ ξ_i - are slack variables representing the margin violations.
- ✓ y_i are the target labels.
- ✓ The term $(1 - \xi_i)$ represents the minimum required margin for each data point.

If a point falls within the margin or on the wrong side, its corresponding value becomes positive, reflecting the amount of violation.

- The objective function of a soft margin SVM combines the margin maximization with a penalty term for margin violations, minimizing: $\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N}\xi_i$

Where:

- ✓ The parameter C in Support Vector Machines serves as a regularization parameter, dictating the balance between the width of the margin and the tolerance for classification errors.
- ✓ N is the number of data points.

The equation combines the objectives of maximizing the margin (represented by the first term) and minimizing the penalty for margin violations (represented by the second term). The regularization parameter (C) controls the trade-off between these objectives. A higher C value prioritizes a wider margin, even if it allows some misclassifications. Conversely, a lower C value allows for more margin violations to achieve a smoother decision boundary.

Advantages of Soft Margin SVM

- ***Robustness to Outliers:*** Soft margin SVM can handle outliers or noisy data more effectively by allowing for some misclassifications. This results in a more robust decision boundary that generalizes better to unseen data.
- ***Applicability to Non-linear Data:*** Unlike hard margin SVM, soft margin SVM can handle non-linearly separable data by implicitly mapping it to a higher-dimensional space using kernel functions. This enables SVM to capture complex decision boundaries.

Disadvantages of Soft Margin SVM

- ***Need for Parameter Tuning:*** The performance of soft margin SVM heavily depends on the choice of the regularization parameter C. Selecting an appropriate value for C requires careful tuning, which can be time-consuming and computationally expensive, especially for large datasets.
- ***Potential Overfitting:*** In cases where the value of C is too large, soft margin SVM may overfit the training data by allowing too many margin violations.

Hard Margin vs Soft Margin in SVM

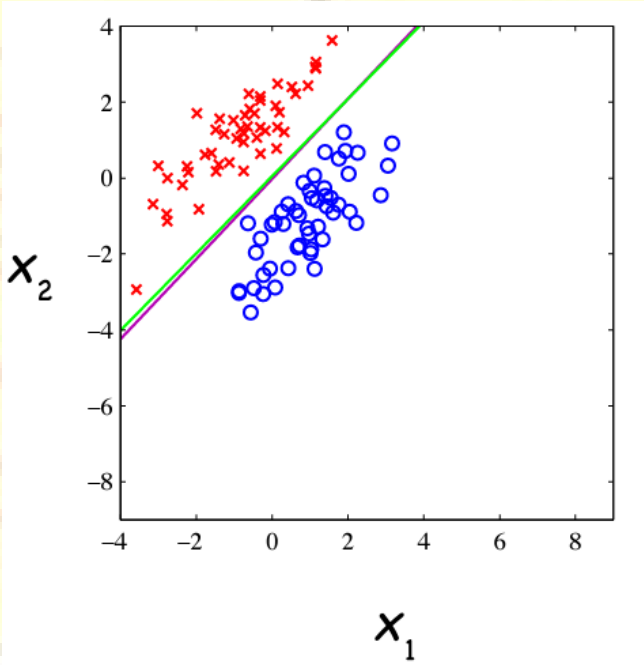
Criteria	Hard Margin	Soft Margin
Objective Function	Maximize margin.	Maximize margin, minimize margin violations.
Handling Noise	Sensitive, requires perfectly linearly separable data	Robust, handles noisy data with margin violations.
Regularization	Not applicable, no regularization parameter	Controlled by regularization parameter C.
Complexity	Simple, computationally efficient	May require more computational resources

Obtaining probabilities from Linear classifiers

- In regression, we are modeling the relationship between a continuous input variable x and a continuous target variable t.
- In classification, the input variable x may still be continuous, but the target variable is discrete.
- In the simplest case, t can have only 2 values:
  e.g., Let t = +1 assigned to C_1
  t = -1 assigned to C_2

Linear Classifier

- The basic idea behind a linear classifier two target classes can be separated by a hyperplane in the feature space. If this can be done without error, the training set is called *linearly separable*.
- As we know, a linear classifier produces scores $\hat{s}(x_i) = w.x_i - t$ that are thresholded at 0 in order to classify examples.
  Example: 2D Input vector x
  Two discrete classes C_1 and C_2



We have already seen linear regression and Ordinary Least Squares (OLS). Let's consider a binary classification problem, and denote target classes to be "+1" (positive examples) and "-1" (negative examples). One of the simplest linear classifiers can be defined using regression as follows:

$$a(x) = sign(w^T x)$$

where,

x – is a feature vector (along with identity);
w – is a vector of weights in the linear model (with bias w_0);
$sign(\bullet)$ – is the signum function that returns the sign of its argument;
$a(x)$ – is a classifier response for x.

Logistic Regression as a Linear Classifier

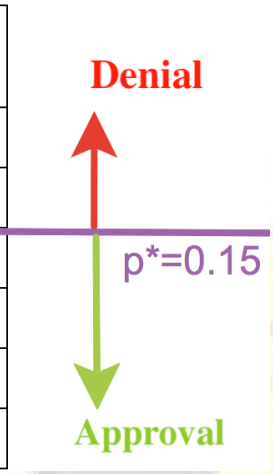
Logistic regression is a special case of the linear classifier, but it has an added benefit of predicting a probability p_+ of referring example x_i to the class "+":

$$p_+ = P(y_i = 1 | x_i, w)$$

Being able to predict not just a response ("+1" or "-1") but the probability of assignment to

class "+1" is a very important requirement in many business problems e.g. credit scoring where logistic regression is traditionally used. Customers who have applied for a loan are ranked based on this predicted probability (in descending order) to obtain a scoreboard that rates customers from bad to good. Below is an example of such a toy scoreboard.

Client	Predicted Default Probability
Mike	0.78
Jack	0.45
Larry	0.13
Kate	0.06
William	0.03
Jessica	0.02



The bank chooses a threshold $\boldsymbol{p}_*$ to predict the probability of loan default (in figure above 0.15) and stops approving loans starting from that value. Moreover, it is possible to multiply this predicted probability by the loan amount to get the expectation of losses from the client, which can also constitute good business metrics (scoring experts may have more to add, but the main gist is this).

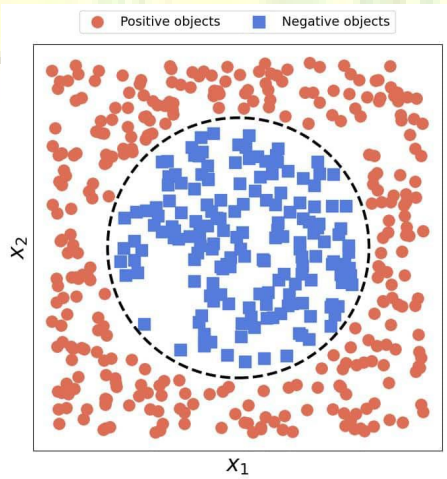
To predict the probability $p_+ \in [0,1]$, we can start by constructing a linear prediction using OLS: $b(x) = w^T x \in \mathbb{R}$. But converting the resulting value to the probability within in the [0, 1] range requires some function $f = \mathbb{R} \to [0,1]$. Logistic regression uses a specific function for this: $\sigma(z) = \frac{1}{1+exp^{-z}}$. Now let's understand what the prerequisites are.

Let's denote the probability of an event *X* as *P(X)*. Then the odds *OR(X)* are determined by $\frac{P(X)}{1-P(X)}$, which is the ratio of the probabilities of whether or not an event will happen. It is obvious that the probability and odds contain the same information, but, while *P(X)* ranges from 0 to 1, *OR(X)* is in the range of 0 to ∞.

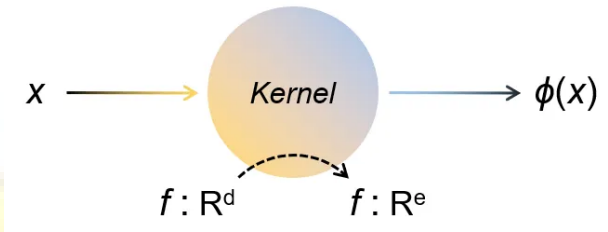
If we calculate the logarithm of *OR(X)* (a logarithm of odds or log probability ratio), it is easy to notice that $\log OR(X) \in \mathbb{R}$. This is what we will use with OLS.

Kernel methods for non-Linearity

- It's easy to fit a straight line to reasonably linear data to find a linear boundary between linearly separable classes. Unfortunately, no straight line can capture a circular boundary.



- As a rule, the more complex and pronounced the non-linear relationships in the data are, the poorer the linear model's performance. Still, we'd like to have models that keep the favorable properties of the linear ones but can handle arbitrarily complex data.
- Kernel Function generally transforms the training set of data so that a non-linear decision surface is able to transform to a linear equation in a higher number of dimension spaces.



- The most popular kind of kernel approach is the ***Support Vector Machine (SVM)***, a binary classifier that determines the best hyperplane that most effectively divides the two groups.
- Other examples of kernel methods include Gaussian kernel, Gaussian kernel Radial Basis Function (RBF) kernel, Sigmoid Kernel, Polynomial kernel, Laplacian kernel and Linear kernel.
- Kernel methods are a class of algorithms used in machine learning to address classification, regression, and other prediction tasks.
- In order to efficiently locate the ideal hyperplane, SVMs map the input into a higher-dimensional space using a kernel function.
- Nonetheless, by incorporating a certain mapping function $\phi(x)$, the feature vectors can be transformed into 3-D feature space. The new data points with 3-D feature vectors $\phi(x) = [x_1, x_2, (x_1^2+x_2^2)]$ can now be using the linear classifier to determine the decision boundary hyperplane. This is the power of feature mapping that can allow us to deal with the more complex data distribution pattern with more expressive ability.

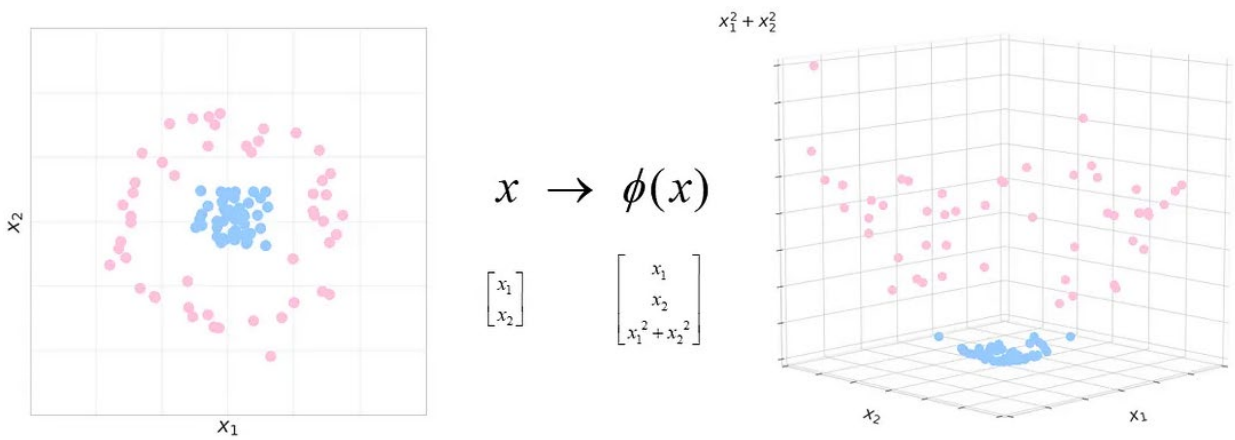


Fig.:- Mapping data points with 2-D feature vectors into 3-D feature vectors

- However, the drawbacks of using $\phi(x)$ directly are that
  - ✓ It is sometimes hard to explicitly construct a $\phi(x)$ directly.
  - ✓ Increase computational power quickly with the increased feature dimensions.
- But the kernel functions can provide an efficient way to solve this.

Kernel Functions

- The idea of kernel functions is to take the inner products between two feature vectors, and evaluate inner products is not computationally costly. We can then exploit only the result of the inner products in our algorithms.

$$\kappa(x_i, x_j) = \emptyset(x_i).\emptyset(x_j) = (x_i \cdot x_j)^2$$

- ✓ ***Polynomial Kernel:*** The polynomial kernel is a kernel function that allows the learning of non-linear models by representing the similarity of vectors (training samples) in a feature space over polynomials of the original variables.

$$\kappa(x_i, x_j) = (x_i \cdot x_j + 1)^p$$

- ✓ ***Gaussian Kernel:*** It is used to perform transformation when there is no prior knowledge about data.

$$\kappa(x_i, x_j) = e^{\left(\frac{-\|x_i - x_j\|^2}{2\sigma^2}\right)}$$

- ✓ ***Gaussian Kernel Radial Basis Function (RBF):*** It is most typically used in support vector machine classification. Same as above kernel function, adding radial basis method to improve the transformation.

$$\kappa(x_i, x_j) = e^{-\gamma \|x_i - x_j\|^2}$$

- ✓ ***Sigmoid Kernel:*** It is primarily used in neural networks and it is similar to the activation function for neurons in a two-layer perceptron model of the neural network.

$$\kappa(x_i, x_j) = tanh(\gamma \cdot x_i^T x_j + r)$$

- ✓ ***Laplace Kernel:*** The Laplacian kernel, also known as the Laplace kernel or the exponential kernel is a non-parametric kernel that can be used to measure the similarity or distance between two input feature vectors. One advantage of the Laplace kernel is its robustness to outliers, as it places less weight on large distances between the input vectors than the Gaussian kernel.

$$\kappa(x_i, x_j) = e^{-\gamma \|x_i - x_j\|}$$

Where, x_i and x_j are the input feature vectors, gamma is a parameter that controls the width of the Laplacian function, and $\|x_i - x_j\|$ is the L1 norm or Manhattan distance between the input vectors.